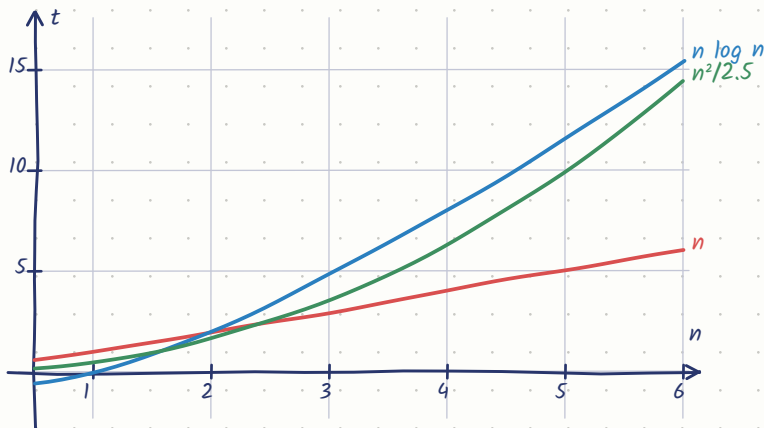


Asymptotics & Graphs

Augii · 01 Aug

Rough idea for today: growth rates first, then how the pipeline fits together, then the messy measurements from the lab.

$f(n) = O(g(n))$ means eventually f sits under some constant multiple of g . The constant doesn't matter, the tail does.



Reading off the picture: quadratic loses at always, even with the friendly constant. Constants shift curves, they never reorder them.

The compiler pipeline we sketched:



with typecheck being the only stage that can reject. Data from the lab runs ~~$n = 10^5$~~ ^{misread the log} $n = 10^4$ elements, three structures:

structure	insert	lookup	notes
sorted array	$O(n)$	$O(\log n)$	cache-friendly
BST (unbalanced)	$O(n)$ worst	$O(n)$ worst	degenerates on sorted input!
hash table	$O(1)$ am.	$O(1)$ am.	resize spikes

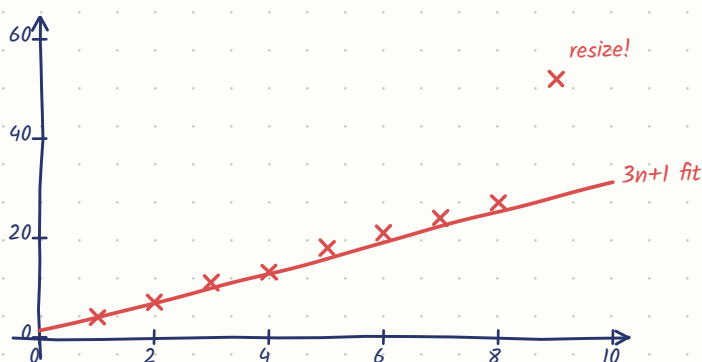
Theorem (Master, case 2)

If $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ and $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.

Proof. Each of the $\log_b n$ levels contributes $\Theta(n^{\log_b a})$ work; sum over levels. ◻

Exam trick: mergesort is exactly case 2 with $a = b = 2$, $f(n) = \Theta(n)$.

Measured scatter, one benchmark run — the outlier at the end is the resize spike from the table above:



◻ re-run with `reserve()` and see if the spike disappears

Careful

Amortized $O(1)$ is a statement about a sequence of operations.

Every operation is fast Any single one can be slow.

